Mercury: Ultra-Fast Language Models Based on Diffusion

Inception Labs
Samar Khanna*, Siddhant Kharbanda*, Shufan Li*, Harshit Varma*, Eric Wang*
Sawyer Birnbaum^, Ziyang Luo^, Yanis Miraoui^, Akash Palrecha^
Stefano Ermon♯, Aditya Grover♯, Volodymyr Kuleshov♯
*^♯ equal core, cross-function, senior contributors listed alphabetically.
hello@inceptionlabs.ai

+ Mercury 2

Presented by:

John Tan Chong Min

Mercury 1 performance (4-5x speedup)

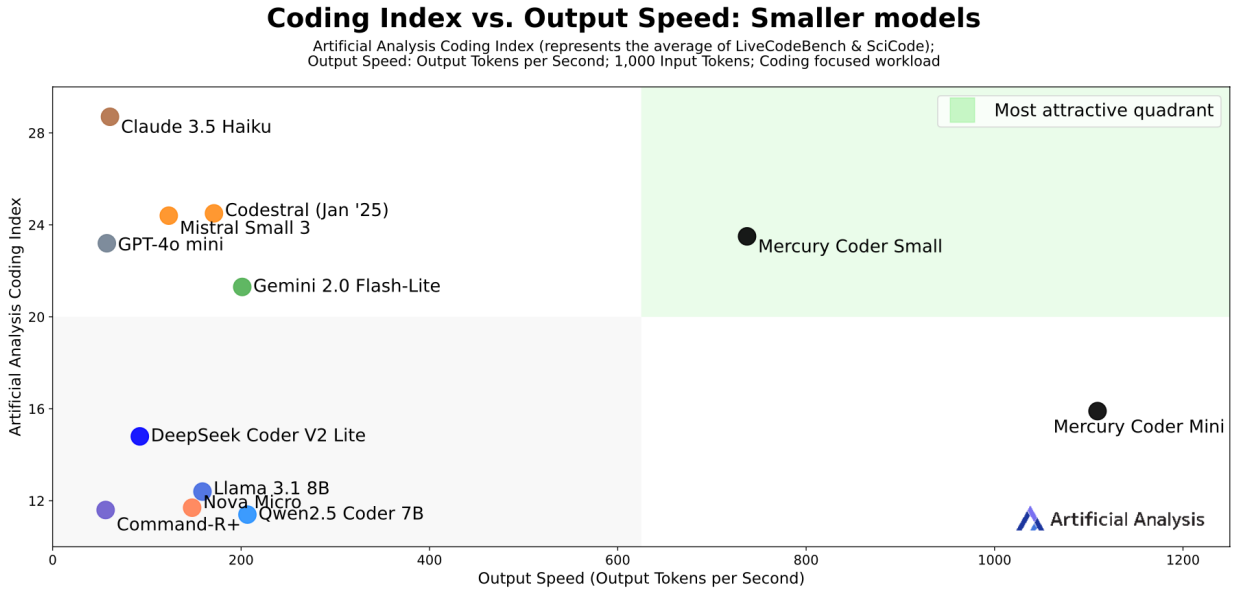


Figure 1: Quality vs. Speed Trade-offs for Mercury Coder models. We find that Mercury Coder models outperform other frontier models by up to 10x in throughput while maintaining comparable quality on challenging code generation benchmarks. Figure taken from third-party evaluations conducted by Artificial Analysis.

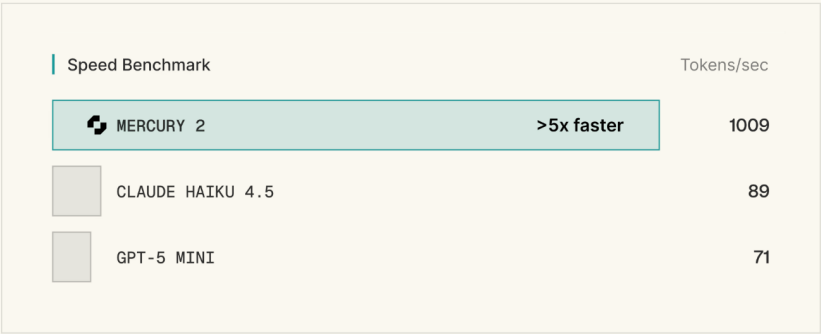
Mercury: Ultra-Fast Language Models Based on Diffusion. Inception Labs. 2025.

Blazing fast speeds (10+x speedup, Mercury 2)

Mercury 2 at a glance

Mercury 2 shifts the quality-speed curve for production deployments:

- **Speed:** 1,009 tokens/sec on NVIDIA Blackwell GPUs
- **Price:** $0.25/1M input tokens · $0.75/1M output tokens
- **Quality**: competitive with leading speed-optimized models
- **Features**: tunable reasoning · 128K context · native tool use · schema-aligned JSON output



https://www.inceptionlabs.ai/blog/introducing-mercury-2

Good scores across benchmarks (Mercury 2)

Model	E2E Latency (Seconds)	GPQA Diamond	LCB	SciCode	IFBench	AIME	TAU
MERCURY 2	**1.7**	**74**	**67**	**38**	**71**	**91**	**53**
GPT-5 NANO (MINIMAL)	4.5	43	47	29	33	27	26
CLAUDE 4.5 HAIKU (NON-REASONING)	5.0	65	51	34	42	39	33
GEMINI 2.5 FLASH-LITE (REASONING)	7.8	71	59	29	53	69	31
GEMINI 3 FLASH (REASONING)	14.4	90	91	51	78	78	80
GEMINI 2.5 FLASH (REASONING)	15.6	79	69	39	50	57	32
GPT-5 MINI (MEDIUM)	22.8	80	69	41	71	48	71
CLAUDE 4.5 HAIKU (REASONING)	23.4	67	62	43	54	84	55

https://www.inceptionlabs.ai/blog/introducing-mercury-2

Surprisingly cheaper costs!

Mercury 2 at a glance

Mercury 2 shifts the quality-speed curve for production deployments:

- **Speed:** 1,009 tokens/sec on NVIDIA Blackwell GPUs
- **Price:** $0.25/1M input tokens · $0.75/1M output tokens
- **Quality**: competitive with leading speed-optimized models
- **Features**: tunable reasoning · 128K context · native tool use · schema-aligned JSON output

Gemini 2.5 Flash

`gemini-2.5-flash`

Try it in Google AI Studio

Our first hybrid reasoning model which supports a 1M token context window and has thinking budgets.

Standard Batch

	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Free of charge	$0.30 (text / image / video) $1.00 (audio)
Output price (including thinking tokens)	Free of charge	$2.50

GPT-5 mini

A faster, cheaper version of GPT-5 for well-defined tasks

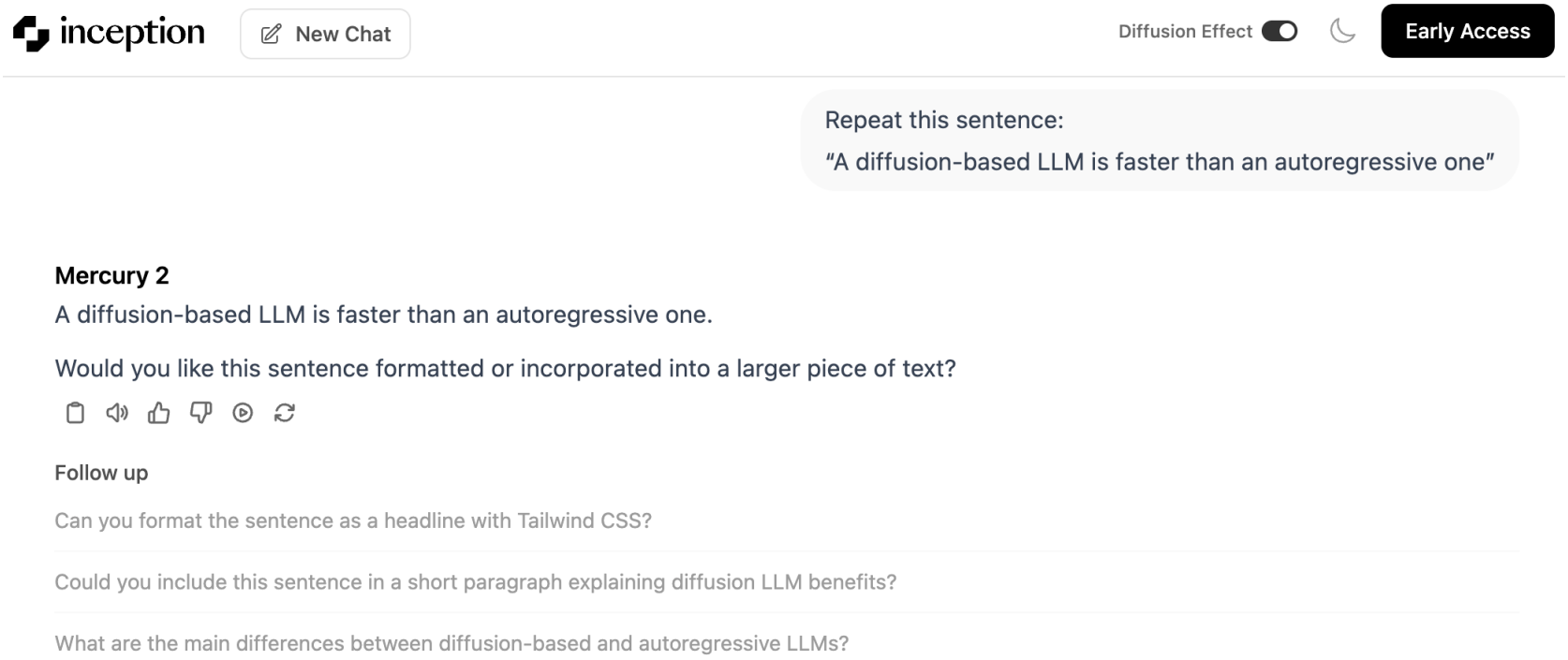
Price

Input:
$0.250 / 1M tokens

Cached input:
$0.025 / 1M tokens

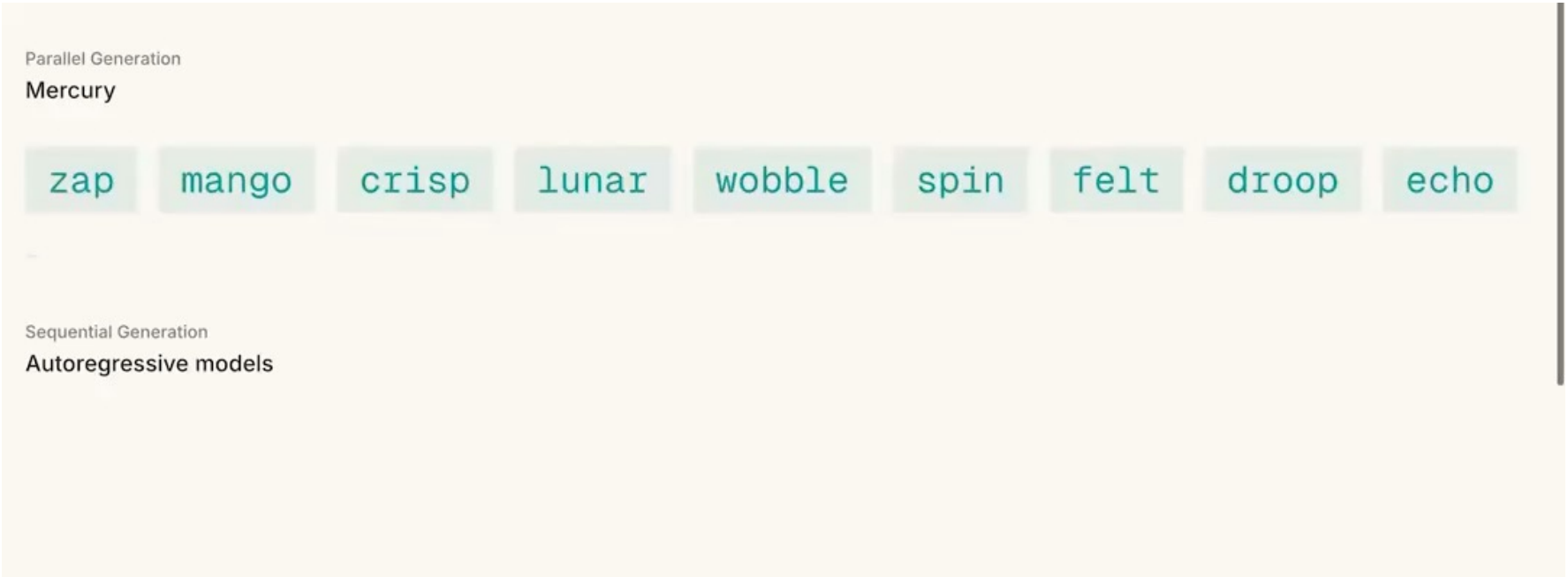
Output:
$2.000 / 1M tokens

Reliability maintained even when it’s faster



https://chat.inceptionlabs.ai/

How diffusion models work



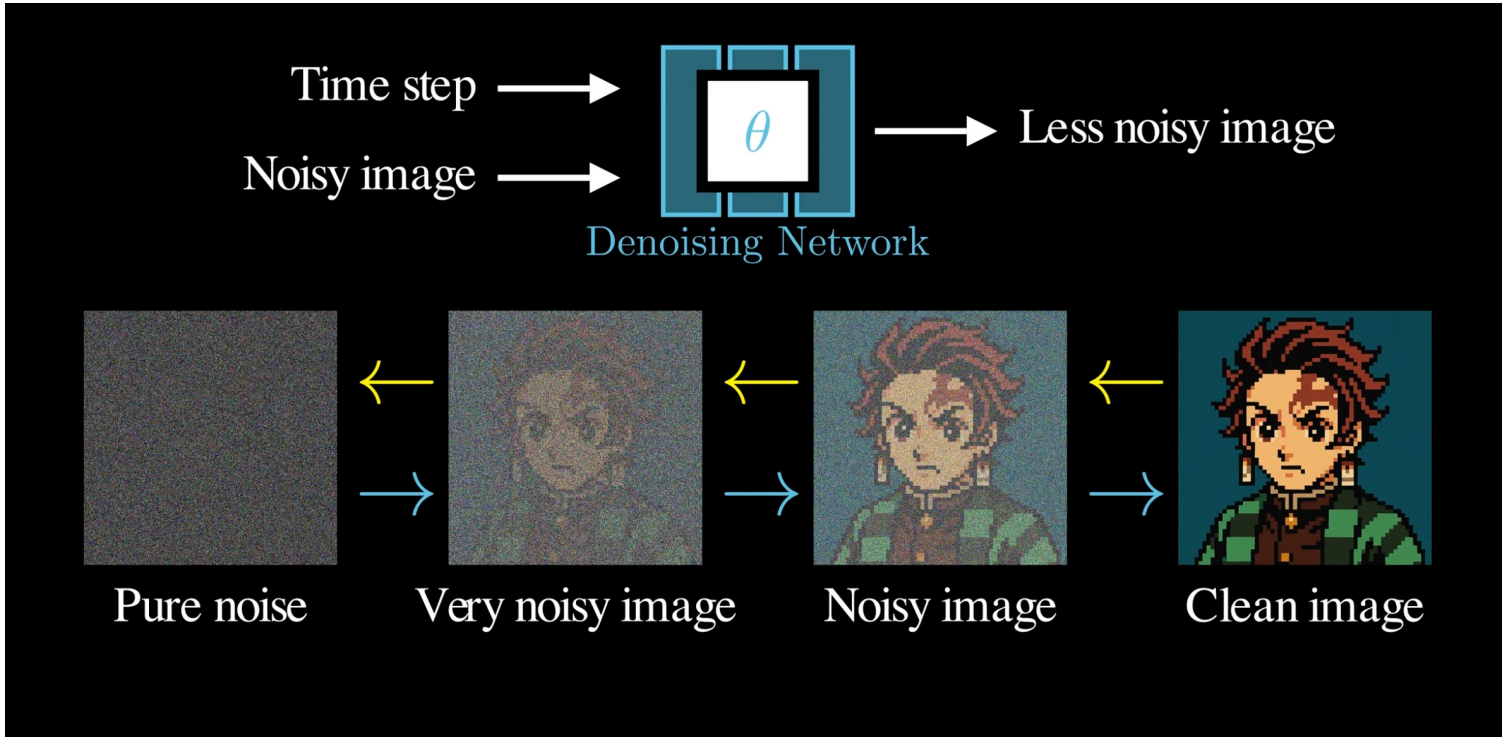
https://www.inceptionlabs.ai/blog/introducing-mercury-2

Some visual illustrations of diffusion

Diffusion Language Models: The Next Big Shift in GenAI. Jia-Bin Huang

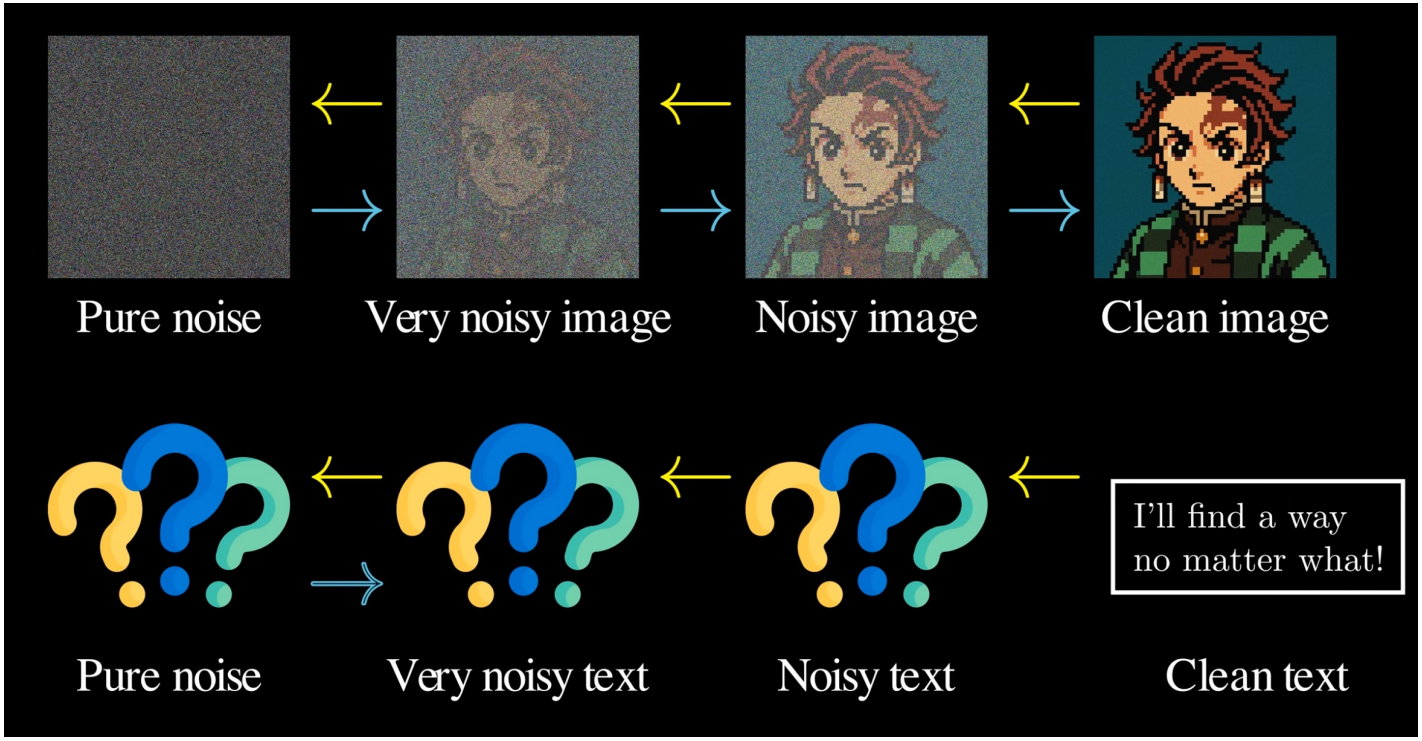
Simple Diffusion Language Models. Sasha Rush

Diffusion Models take noise and iteratively refine to an end product



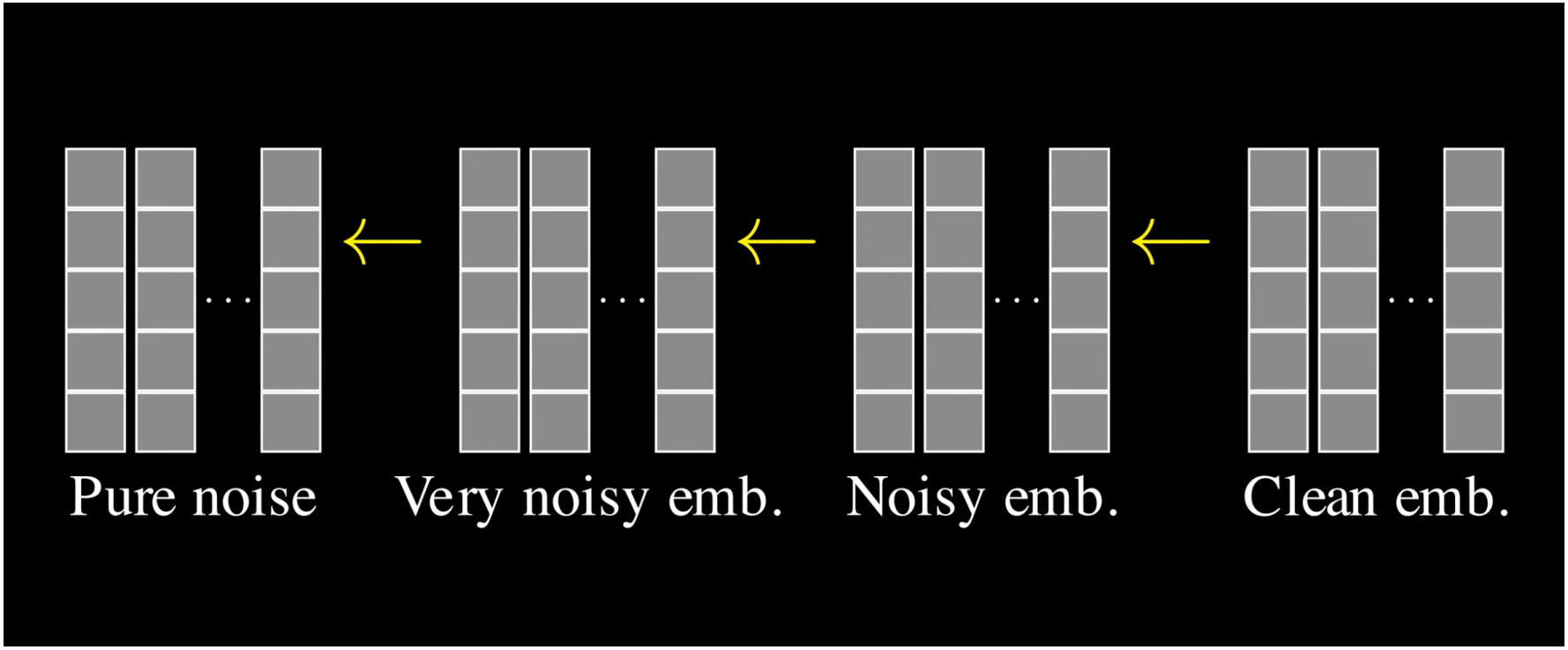
Diffusion Language Models: The Next Big Shift in GenAI. Jia-Bin Huang. https://www.youtube.com/watch?v=8BTOoc0yDVA

Diffusion can be applied to text as well (multimodal too actually)



Diffusion Language Models: The Next Big Shift in GenAI. Jia-Bin Huang. https://www.youtube.com/watch?v=8BTOoc0yDVA

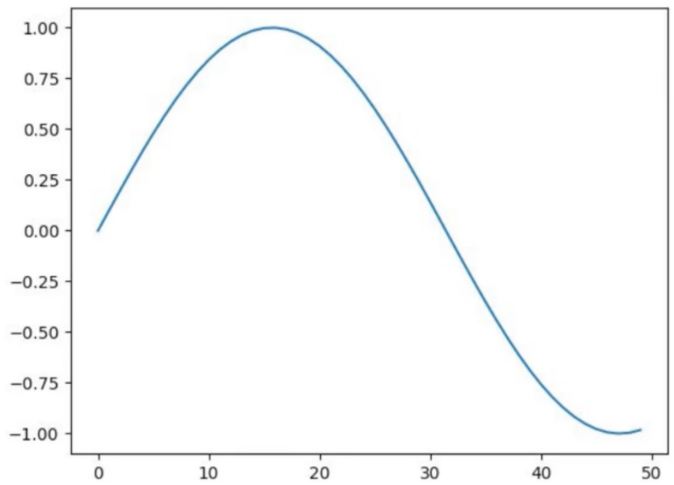
Embedding-based diffusion



Diffusion Language Models: The Next Big Shift in GenAI. Jia-Bin Huang. https://www.youtube.com/watch?v=8BTOoc0yDVA

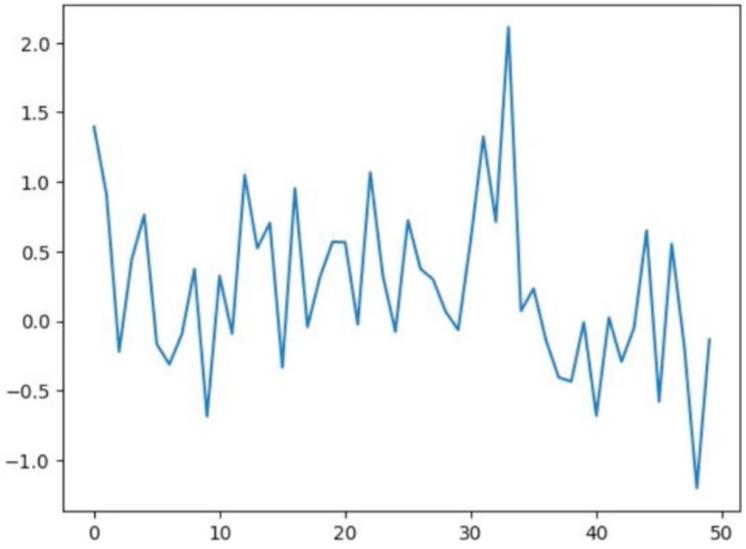
Heatmap view of Embedding Vector

$$x \sim p_\theta(x)$$



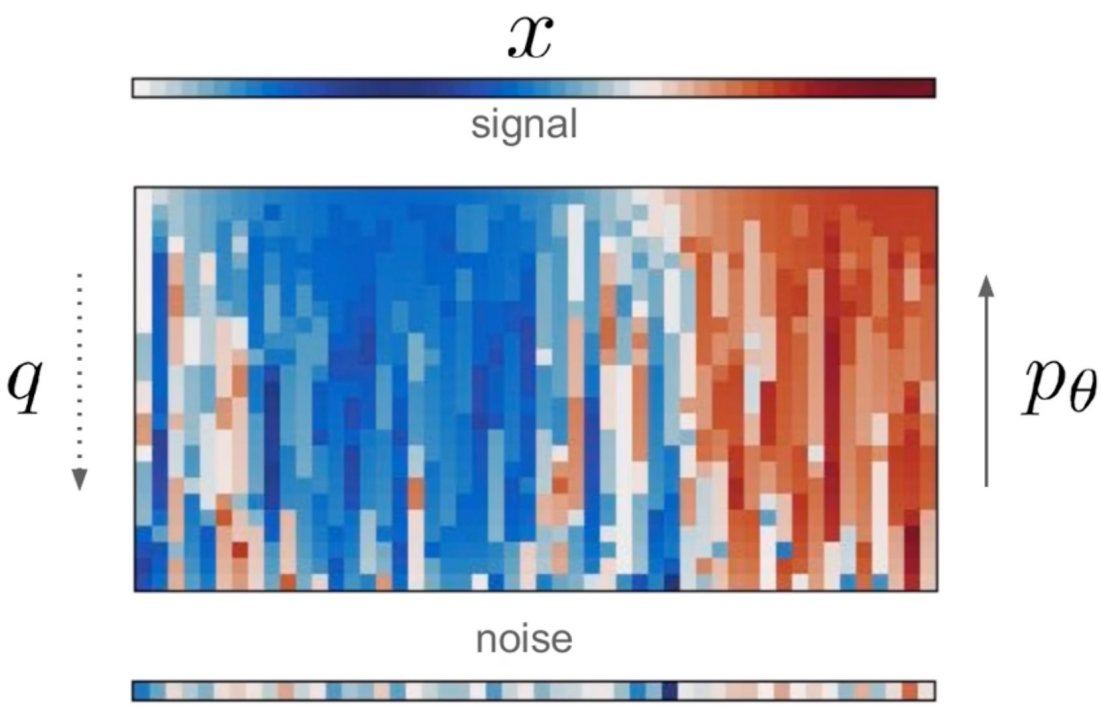
Simple Diffusion Language Models. Sasha Rush. https://www.youtube.com/watch?v=WjAUX23vgfg

Heatmap View of Noisy Vector



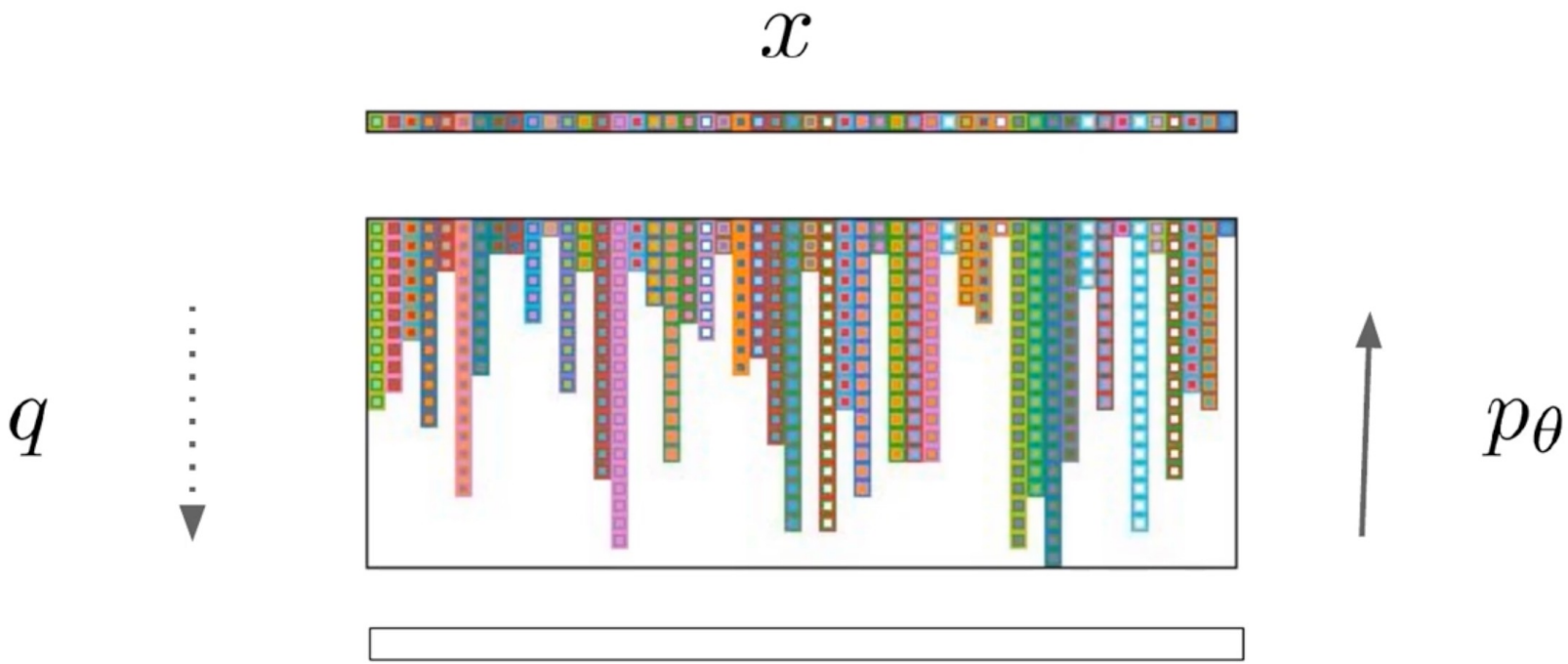
Simple Diffusion Language Models. Sasha Rush. https://www.youtube.com/watch?v=WjAUX23vgfg

q makes signal become noise; ***p*** reverses it



Simple Diffusion Language Models. Sasha Rush. https://www.youtube.com/watch?v=WjAUX23vgfg

Unmask some tokens (even remask) at each denoising time step

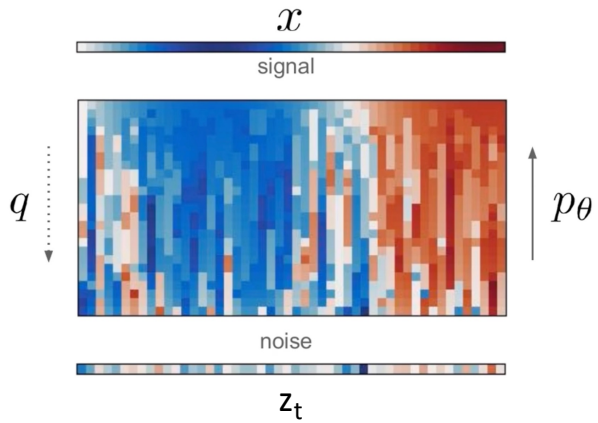


Simple Diffusion Language Models. Sasha Rush. https://www.youtube.com/watch?v=WjAUX23vgfg

Some Technical Details

Math

More formally, we define our diffusion models via a pair of forward and reverse processes. The forward or noising process q starts from clean data $\mathbf{x} \in \mathcal{X}$ (a sequence of natural language tokens, e.g., a sequence of words) and defines a set of latent variables $\mathbf{z}_t \in \mathcal{X}$ over time steps $t = 1, ..., T$ via a Markov process denoted as $q(\mathbf{z}_t|\mathbf{z}_{t-1})$. The latents $\mathbf{z}_t$ represent increasingly noisy versions of $\mathbf{x}$, and the final $\mathbf{z}_T$ are designed to be distributed according to a known prior noise distribution $p(\mathbf{z}_T)$. The reverse or denoising process p generates data by first sampling $\mathbf{z}_T \sim p(\mathbf{z}_T)$ and then applying a model $p(\mathbf{z}_{t-1}|\mathbf{z}_t)$ to iteratively denoise the data. This procedure defines a probability distribution $p(\mathbf{x})$.



TL;DR:

q() makes data noisy
p() makes data clean
x is clean data
z_t is noisy data

Mercury: Ultra-Fast Language Models Based on Diffusion. Inception Labs. 2025.

Math

The model p is defined by learned parameters θ, hence we denote it by p_θ. The parameters are chosen to minimize a loss that fits p to reverse q. In practice, this can be achieved by first learning a denoising model, i.e., by minimizing

$$\mathcal{L}(\mathbf{x}) = -\mathbb{E}_t \left[\gamma(t) \cdot \mathbb{E}_{\mathbf{z}_t \sim q} \log p_\theta(\mathbf{x}|\mathbf{z}_t) \right],$$

Reconstruction loss:
Given corrupted data $\boldsymbol{z_t}$,
how good are you at reconstructing $\boldsymbol{x}$

where $\gamma(t) \geq 0$ is a user-specified function that assigns a weight to each noise level and $p_\theta(\mathbf{x}|\mathbf{z}_t)$ is a distribution over clean data $\mathbf{x}$ given noisy data $\mathbf{z}_t$. The denoiser can then be used for generation e.g., by defining $p_\theta(\mathbf{z}_{t-1}|\mathbf{z}_t) = \sum_x q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}) p_\theta(\mathbf{x}|\mathbf{z}_t)$.

Weight on noise $\boldsymbol{\gamma(t)}$:
- More weight on low noise ⇒ better fine detail / local correction
- More weight on high noise ⇒ better global reconstruction from heavily corrupted input

Mercury: Ultra-Fast Language Models Based on Diffusion. Inception Labs. 2025.

Architecture

Architecture Inception Mercury models are based on a Transformer architecture [40]. Note that this choice of architecture is orthogonal to the fact that the Mercury models are diffusion-based. Diffusion implies specific training and generation algorithms, but does not pose constraints on the architecture of neural network that is trained. For example, a dLLM could also be based on a recurrent architecture [32, 17]. This is analogous to architecture choices for image diffusion models, in which the denoising network can also be parameterized with a U-Net [19] or a transformer [31]. Relying on a Transformer architecture has a number of advantages. It allows Mercury models to benefit from efficient implementations of low-level primitives, and it simplifies hyper-parameter search and optimization.

TL;DR:
Using Transformer as opposed to U-Net for diffusion is probably a better choice,
as Diffusion Transformers (DiT) have been better than U-Net at image regeneration

For more info, read "Scalable Diffusion Models with Transformers. Peebles et. al. 2023."

Mercury: Ultra-Fast Language Models Based on Diffusion. Inception Labs. 2025.

Other Details

Fine-tuning and Alignment Inception Mercury Models can benefit from further pre-training, fine-tuning and alignment on downstream datasets via RLHF [30] or DPO [33] techniques to improve downstream performance. The key change for all stages is to replace the autoregressive loss with a denoising diffusion loss.

Context Length Inception Mercury models support a context length of up to 32,768 tokens out of the box and up to 128k tokens with context extension approaches. This protocol follows standard training recipes used for developing language models [16, 42, 26].

Mercury: Ultra-Fast Language Models Based on Diffusion. Inception Labs. 2025.

Questions to Ponder

- How do diffusion-based approaches know the number of tokens to output?
- Is a diffusion language model more sample efficient than a transformer?
- How can we diffuse multimodal embedding tokens?